

Monologue on Program Design and Computer Languages

Group 5151003/40001 — Totskiy Vjatseslav

Step 1. Introduction

Edsger Dijkstra once said: "The tools we use have a profound influence on our thinking abilities." Today there are hundreds of computer languages, and each one shapes how a programmer solves problems. *Firstly*, I will explain the traits of a programming language; *moreover*, I will talk about program design; *finally*, I will discuss the history of programming languages.

Step 2. Important traits of a programming language

2.1. *To begin with*, a programming language is an artificial language used **to perform computation** or **to control external devices**. Its main purpose is **to allow humans to communicate instructions to machines** through clear **constructs**. *Furthermore*, every language lets programmers **define and manipulate data structures** and **control the flow of execution**. *For example*, the theory of computation **classifies languages by the computations they are capable of expressing**: all **Turing complete languages** share this **expressive power**, while a **formal grammar** like BNF is not a full programming language.

2.2. *Furthermore*, programming languages are used in different contexts: *on the one hand*, a program can **be executed in a batch process without human interaction**, like a build script on a server. *On the other hand*, a user can type commands in an interactive session of an interpreter, which we call a scripting language. *As for* my own experience, I write Python scripts in QA automation and use Solidity for smart contracts. *Consequently*, the context — server, browser, or blockchain — decides which language is right for the job.

Step 3. Program design

3.1. Programming languages differ from natural languages because they require **a greater degree of precision and completeness**. *However*, in English or Russian we can be ambiguous and still expect our intent to be understood, but computers do exactly what they are told to do. *For instance*, the language definition, the program, and its inputs must fully specify the external behaviour that occurs at runtime. *As a result*, a missing semicolon can break the whole program, which would never happen in human conversation.

3.2. Many languages have been designed from scratch, *then* **altered to meet new needs**, **combined with other languages**, and eventually some **fall into disuse**. The need comes from **the diversity of contexts** in which they are used, and programmers **range in expertise** from novices to experts. *Moreover*, languages **undergo modification** constantly — Python 2 was replaced by Python 3. *Consequently*, the main trend is **to solve problems using a higher level of abstraction**, so programmers are less **tied to the underlying hardware**.

Step 4. History of programming languages

4.1. The first programming languages actually predate the modern computer. *For example*, in the 19th century, programmable looms were already **domain-specific programming languages** that used physical patterns **to direct mechanical processing**. *Later*, punch cards were used **to encode data** and control early machines. *Furthermore*, in the 1930s and 1940s, lambda calculus and Turing machines provided **a formal system in mathematical**

logic for expressing algorithms, and these ideas became the foundation of modern programming.

4.2. *In the 1940s*, the first **electrically powered digital computers** appeared, and Konrad Zuse's Plankalkül became the first high-level language. The 1950s brought **machine code** programming on UNIVAC, which was superseded by assembly — a **low-level programming language** close to the hardware. *Furthermore*, the late 1950s introduced **high-level programming languages** like FORTRAN, LISP, and COBOL, which **use forms resembling English**. *As for* modern times, my main languages are Python and TypeScript, plus some C++ at university.

Step 5. Creative Thinking

Apart from traditional programming, I want to mention AI-assisted code generation with models like GitHub Copilot and Claude. *Not only* can these tools translate plain English into working code, *but also* they can debug, refactor, and write tests. *Consequently*, we are moving towards a new "generation" of programming languages where the source is English and the compiler is AI. As a future security engineer, I find this exciting but also concerning, because AI-generated code can hide subtle vulnerabilities.

Step 6. Conclusion

To sum up, a programming language is a tool with its own **constructs**, **expressive power**, and contexts of use, and program design requires more precision than natural language. *Additionally*, the history of computer languages shows a clear path from punch cards and **machine code** to **high-level programming languages** that **use forms resembling English**. *In conclusion*, as AI continues to reshape software development, programming languages will keep changing — but the idea of communicating instructions to machines will stay the same.

Yellow = Active Vocabulary, *Green/Bold Italic = Linkers*

Active Vocabulary: 29, Linkers: 20. Total: 691 words.